Object Oriented Programming(C++)
(CSE 1211)

Inheritance

Md. Mahfuzur Rahman
Lecturer,
Prime University

Inheritance

1. What is Inheritance ?

   The process of obtaining the data members and functions from one class to another class is known as **inheritance**.

2. What are the Importance of inheritance ?
   - ✓ Code reusability
   - ✓ Application development time is less.
   - ✓ Application take less memory.
   - ✓ Application execution time is less.

Inheritance

Inheritance in Object-Oriented Programming (OOP) is a fundamental concept that allows a new class (known as the derived class or child class) to inherit properties and behaviors from an existing class (known as the base class or parent class). This mechanism promotes code reusability and helps establish a hierarchical relationship between classes, mirroring real-world "is-a" relationships (e.g., a "Dog is a" "Mammal").

In Object-Oriented Programming (OOP), inheritance is a mechanism that allows a new class (derived class or subclass) to inherit properties (attributes) and behaviors (methods) from an existing class (base class or superclass), establishing an "IS-A" relationship

C++ Inheritance

- In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.
- In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

Advantage of C++ Inheritance

Code reusability: Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

Key Concepts

- **Base Class (Superclass/Parent Class):** The existing class whose properties and behaviors are inherited.
- **Derived Class (Subclass/Child Class):** The new class that inherits from the base class and gains access to its features.
- **"IS-A" Relationship:** A fundamental concept in inheritance, signifying that the derived class is a more specific type of the base class (e.g., a "Dog is an Animal").

How Inheritance Works

- **Code Reuse:** Common attributes and methods are defined in the base class and automatically available to all its derived classes, eliminating the need to rewrite the same code.
- **Class Hierarchy:** Inheritance creates a natural hierarchy of classes, where a more general class serves as a template for more specific ones.
- **Extending Functionality:** Derived classes can add new, unique attributes and methods to customize their behavior.
- **Overriding Methods:** A derived class can provide its own specific implementation for a method inherited from the base class.

Benefits of Inheritance

Code Reusability: Reduces code duplication, saving development time and effort.

Modularity: Improves code organization by grouping related functionalities into a clear hierarchy.

Maintainability: Changes made to the base class are automatically reflected in its subclasses, simplifying updates.

Extensibility: Allows for easy addition of new features to an existing codebase without modifying the original class.

Inheritance

```
class Person
{
    public:
        string name;
        int age;

        void display1()
        {
            cout << "Name : " << name <<endl;
            cout << "Age : " << age <<endl;
        }
};
```

```
class Student
{
    public:
        string name;
        int age;
        int id;

        void display2()
        {
            cout << "Name : " << name <<endl;
            cout << "Age : " << age <<endl;
            cout << "Id : " << id <<endl;

        }
};
```

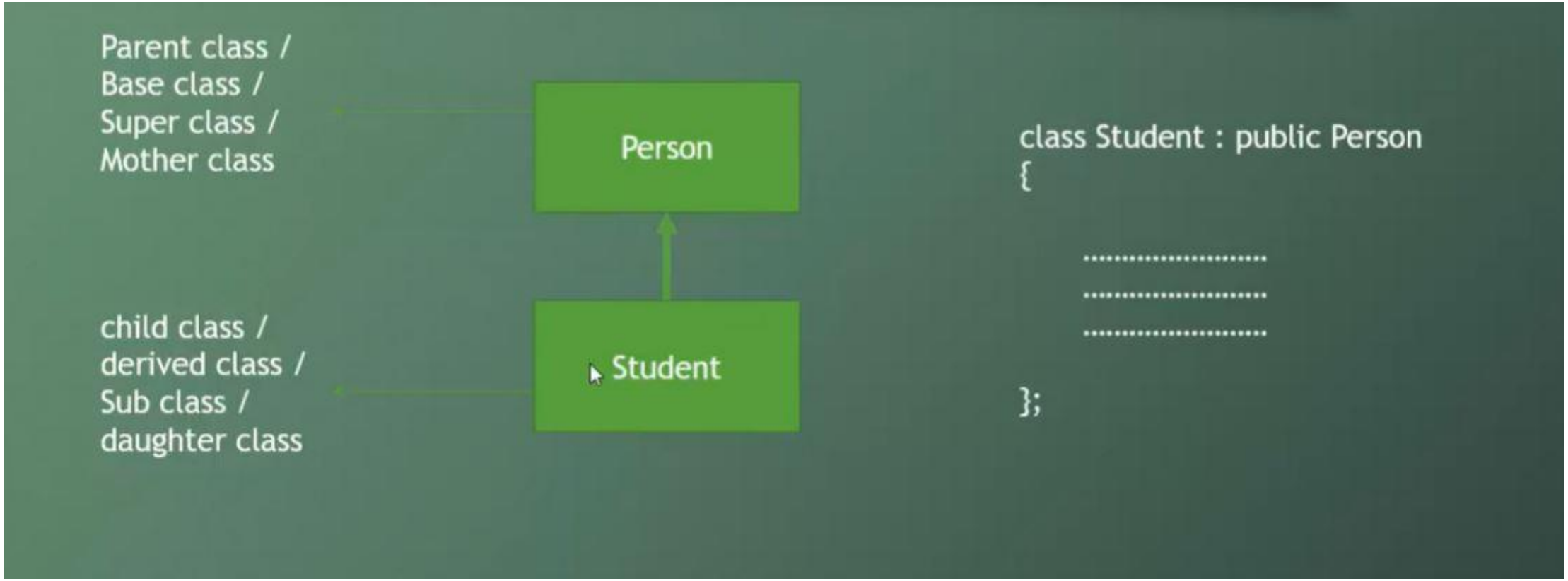
Inheritance

```
class Person
{
    public:
        string name;
        int age;

        void display1()
        {
            cout << "Name : " << name <<endl;
            cout << "Age : " << age <<endl;
        }
};
```

```
class Student : public Person
{
    public:
            int id;
        //string name;
        // int age;
        // void display1()
            void display2()
            {
                cout << "Id : " << id <<endl;
                display1();
            }
};
```

Inheritance



Inheritance

```cpp
#include<iostream>
using namespace std;

class Person
    {
    public:
        int id;
        string name;

        void display1()
        {
            cout << "ID :" << id << endl;
            cout << "Name :" << name << endl;
        }
    };

class Student : public Person
{
public:

    int Age;
    void display2()
        {
            cout << "ID :" << id << endl;
            //cout << "Name :" << name << endl;
            //cout << "Age :" << Age << endl;
            display1();
        }

};
```

```cpp
int main()
{
    Student obj;
    obj.id = 101;
    obj.name = "esha";
    obj.Age = 20;

    obj.display2();
}
```

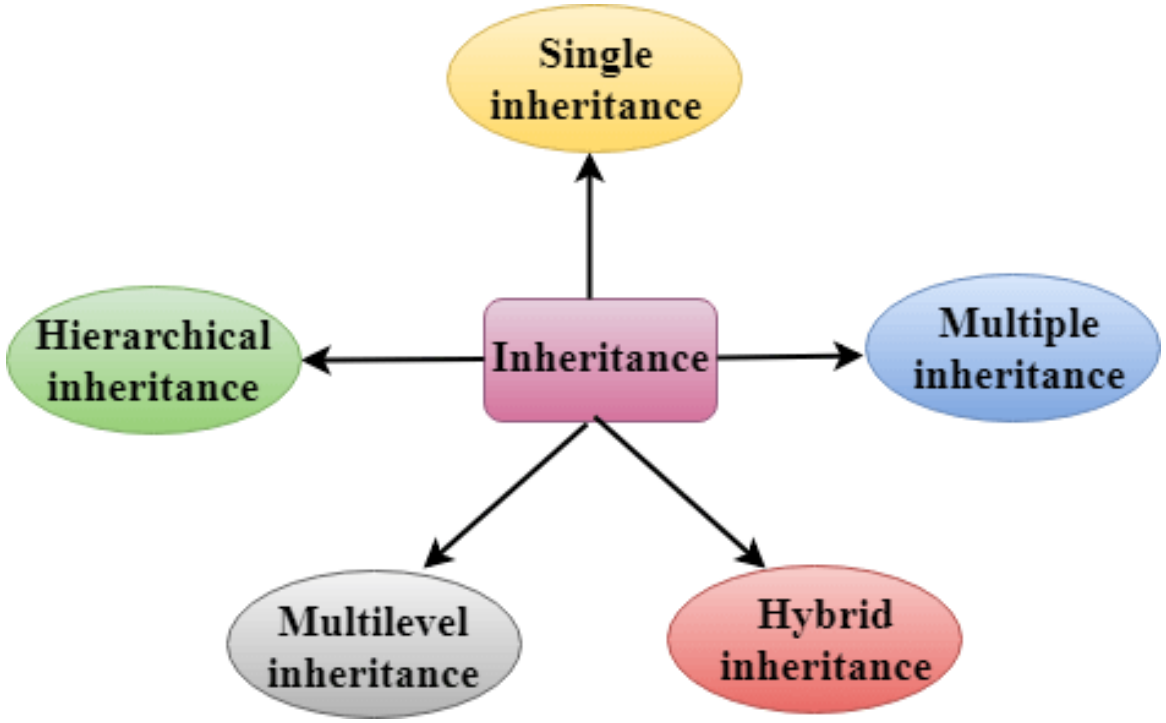
Why Use Inheritance in OOP?

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.
- **Method Overriding:** Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.
- **Abstraction:** The concept of abstraction where we do not have to provide all details, is achieved through inheritance. Abstraction only shows the functionality to the user.

Types Of Inheritance in C++

The inheritance can be classified on the basis of the relationship between the derived class and the base class. In C++, we have **5 types of inheritances:**

- Single inheritance
- Multilevel inheritance
- Multiple inheritance
- Hierarchical inheritance
- Hybrid inheritance



Derived Classes

A Derived class is defined as the class derived from the base class.

The Syntax of Derived class:

```
class derived_class_name : access specifier base_class_name
{
    // body of the derived class.
}
```

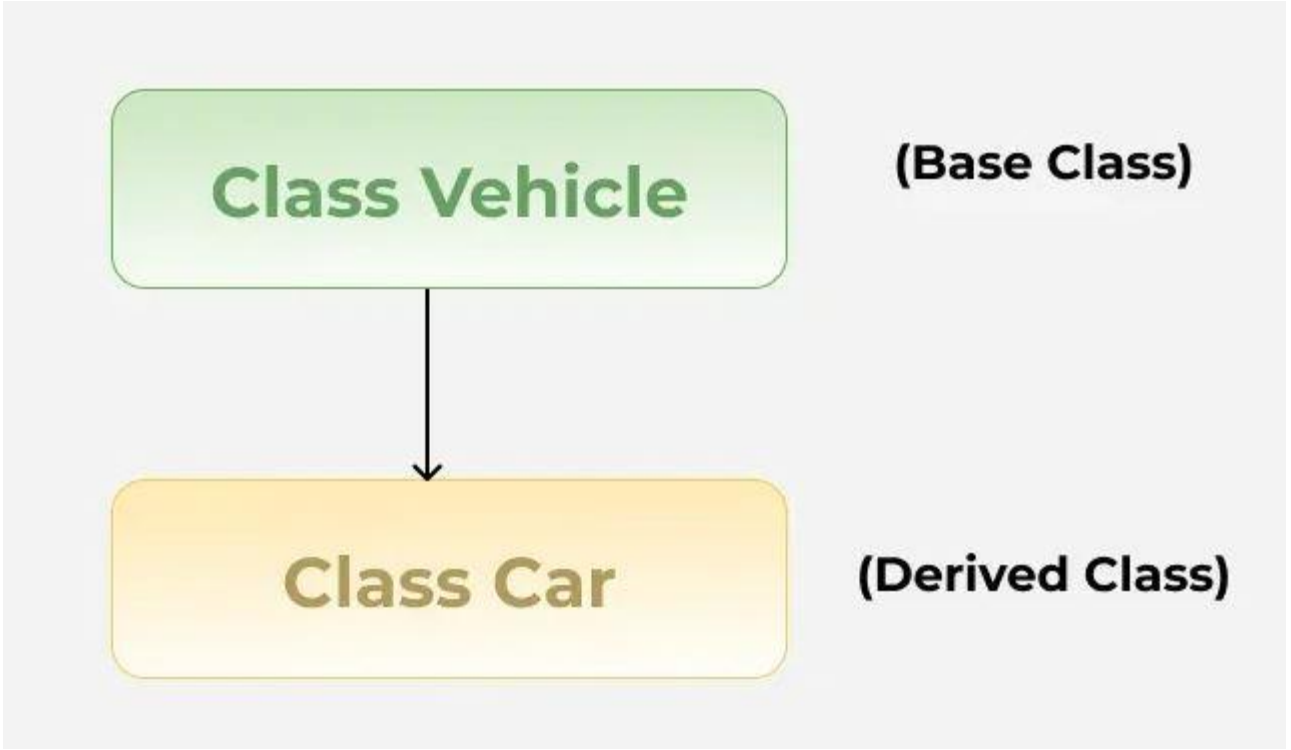
Where,

derived_class_name: It is the name of the derived class.

base_class_name: It is the name of the base class.

Single Inheritance

In single inheritance, a class is allowed to inherit from only one class. i.e. one base class is inherited by one derived class only.



Single Inheritance

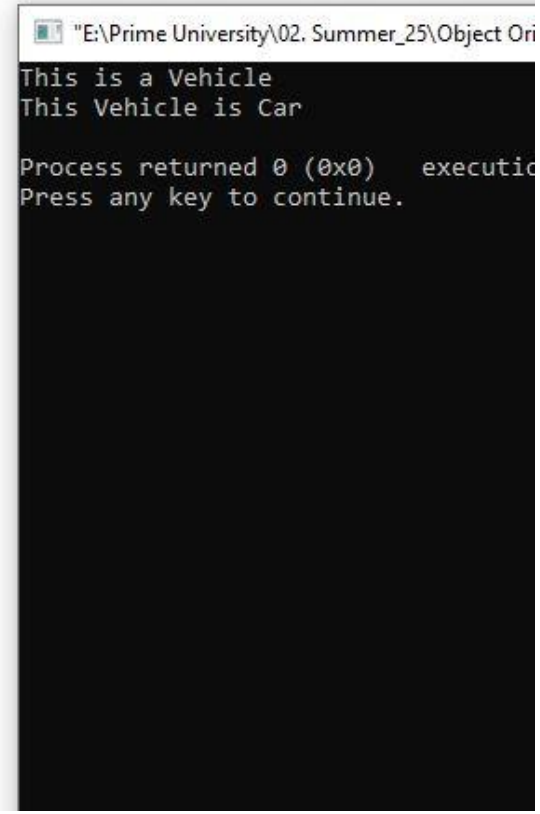
```cpp
#include <bits/stdc++.h>
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

class Car : public Vehicle {
public:
    Car() {
        cout << "This Vehicle is Car"<< endl;
    }
};

int main() {

    Car obj;
    return 0;
}
```



Inheritance

```cpp
#include<iostream>
using namespace std;

class Person
    {
    public:
        int id;
        string name;

        void display1()
        {
            cout << "ID :" << id << endl;
            cout << "Name :" << name << endl;
        }
    };

class Student : public Person
{
public:

    int Age;
    void display2()
        {
            cout << "ID :" << id << endl;
            //cout << "Name :" << name << endl;
            //cout << "Age :" << Age << endl;
            display1();
        }

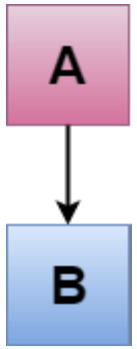
};
```

```cpp
int main()
{
    Student obj;
    obj.id = 101;
    obj.name = "esha";
    obj.Age = 20;

    obj.display2();
}
```

C++ Single Inheritance

Single inheritance is defined as the inheritance in which *a derived class is inherited from the only one base class.*
Where 'A' is the base class, and 'B' is the derived class.



Where 'A' is the base class, and 'B' is the derived class.

C++ Single Level Inheritance Example:

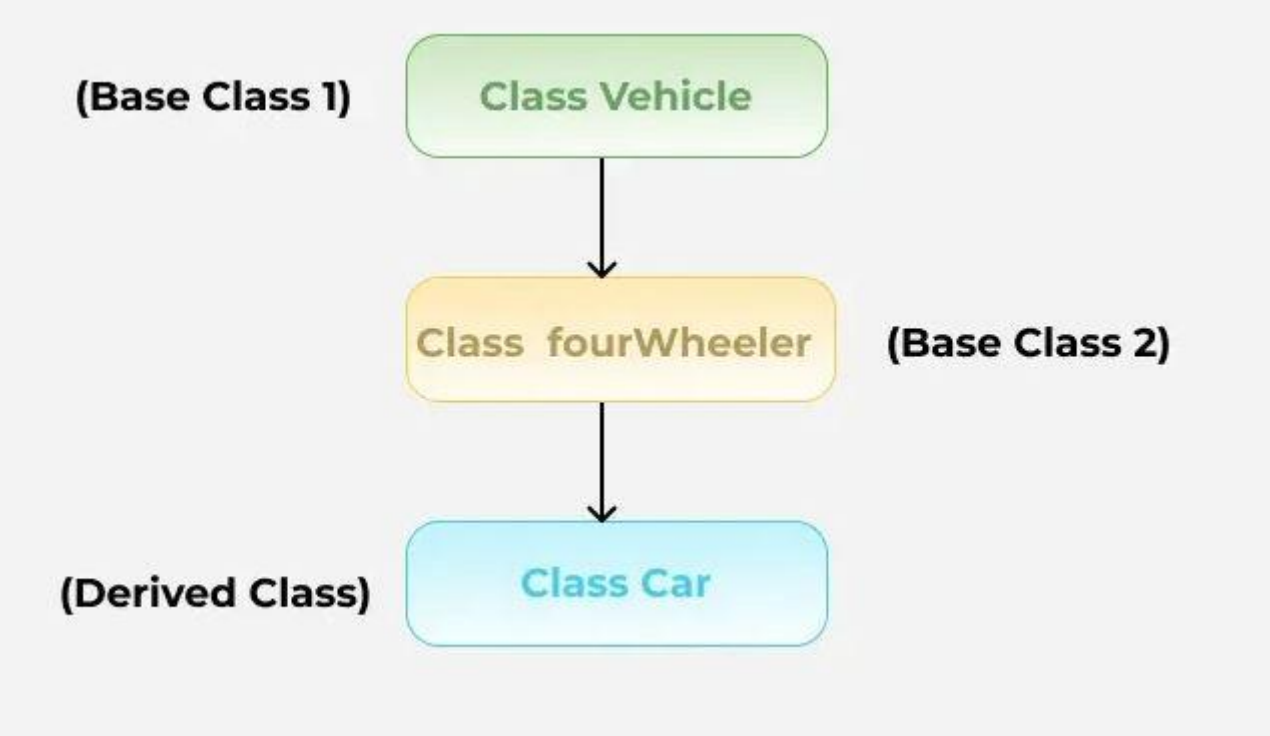
```
#include <iostream>
using namespace std;
 class Account {
   public:
   float salary = 60000;
 };
   class Programmer: public Account {
   public:
   float bonus = 5000;
   };
int main(void) {
     Programmer p1;
     cout<<"Salary: "<<p1.salary<<endl;
     cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Output:

Salary: 60000
Bonus: 5000

Multilevel Inheritance

In multilevel inheritance, a derived class is created from another derived class and that derived class can be derived from a base class or any other derived class. There can be any number of levels. For example, a vehicle can be a four-wheeler, and a four-wheeler vehicle can be a car.



Multilevel Inheritance

```cpp
#include <bits/stdc++.h>
using namespace std;

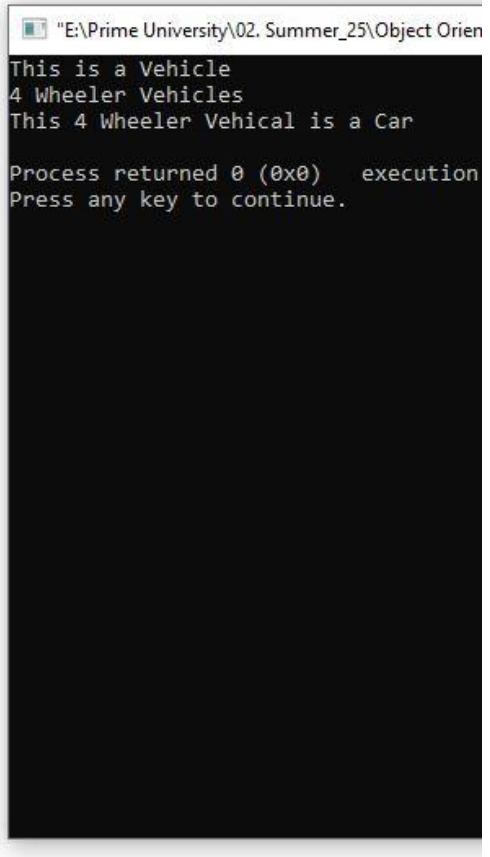
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

class fourWheeler : public Vehicle {
public:
    fourWheeler() {
        cout << "4 Wheeler Vehicles"<< endl;
    }
};

class Car : public fourWheeler {
public:
    Car() {
        cout << "This 4 Wheeler Vehical is a Car" << endl;
    }
};

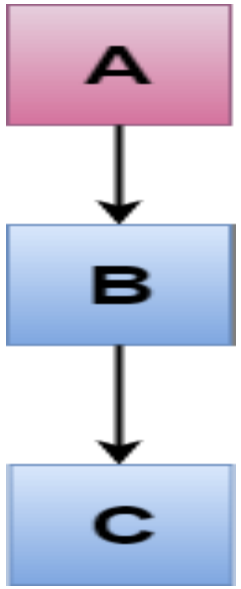
int main() {

    Car obj;
    return 0;
}
```



C++ Multilevel Inheritance

Multilevel inheritance is a process of deriving a class from another derived class.



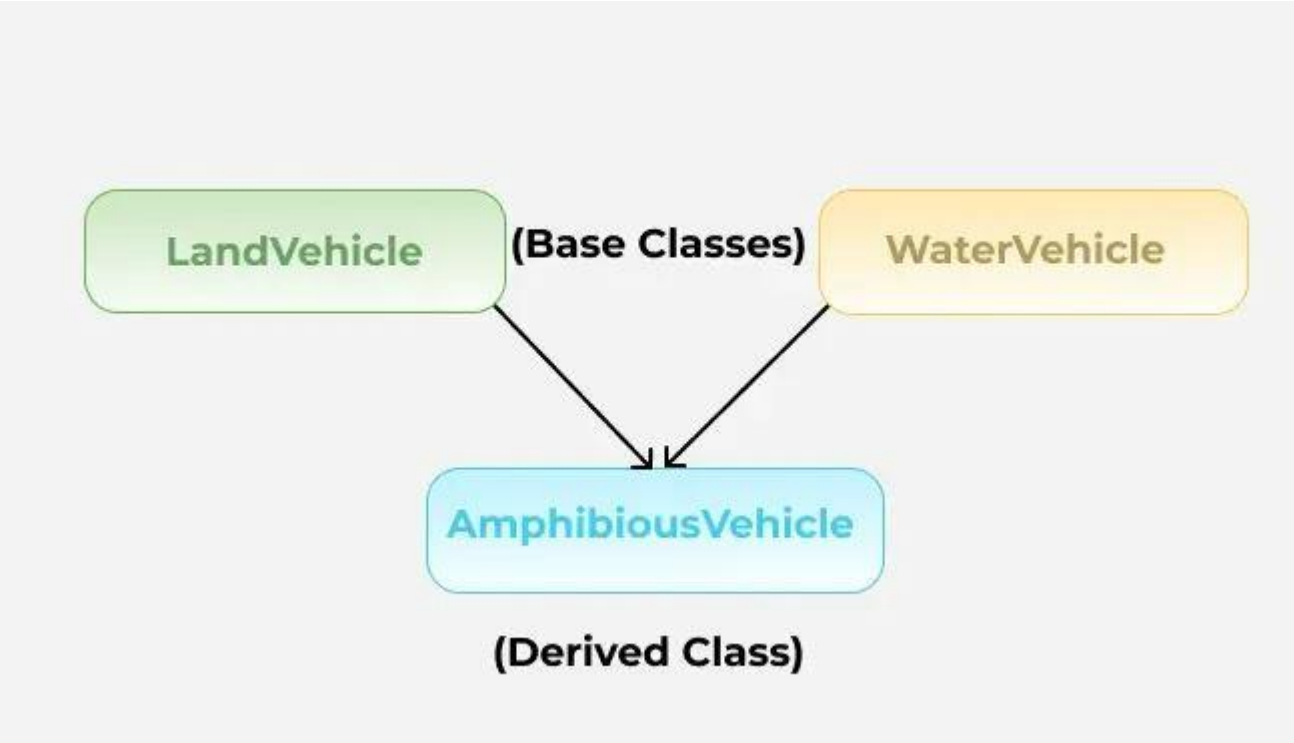
C++ Multi Level Inheritance Example

When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.

```
#include <iostream>
using namespace std;
 class Animal {
   public:
 void eat() {
    cout<<"Eating..."<<endl;
 }
   };
   class Dog: public Animal
   {
      public:
     void bark(){
    cout<<"Barking..."<<endl;
     }
   };
class BabyDog: public Dog
   {
      public:
     void weep() {
    cout<<"Weeping...";
     }
   };
int main(void) {
    BabyDog d1;
    d1.eat();
    d1.bark();
     d1.weep();
     return 0;
}
```

Multiple Inheritance

Multiple Inheritance is a feature of C++ where a class can inherit from more than one class. i.e one **subclass** is inherited from more than one **base class.**



Multiple Inheritance

```cpp
#include <bits/stdc++.h>
using namespace std;

class LandVehicle {
public:
    LandVehicle() {
        cout << "This is a LandVehicle"<< endl;
    }
};

class WaterVehicle {
public:
    WaterVehicle() {
        cout << "This is a WaterVehicle"<< endl;
    }
};

class AmphibiousVehicle : public WaterVehicle, public LandVehicle {
  public:
    AmphibiousVehicle() {
        cout << "This is an AmphibiousVehicle"<< endl;
    }
};

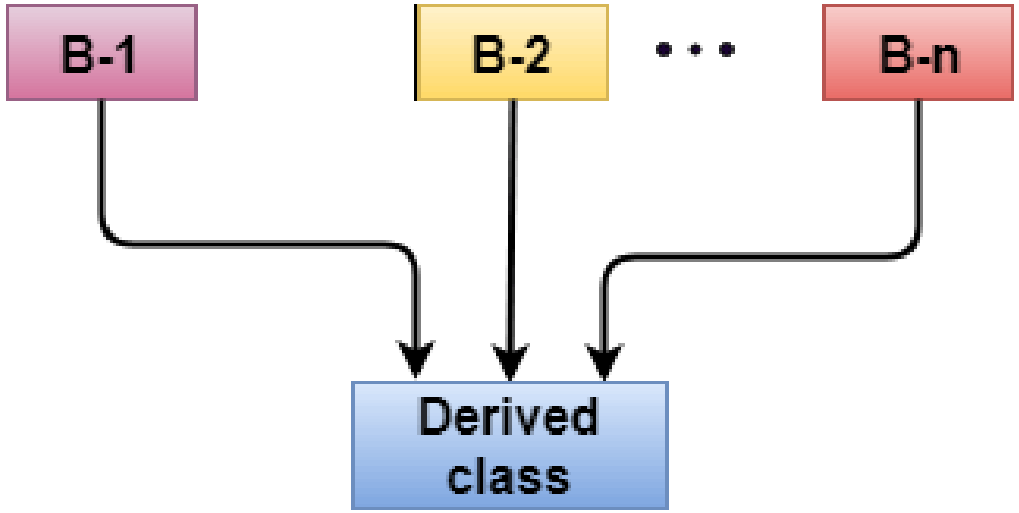
int main() {

    AmphibiousVehicle obj;
    return 0;
}
```



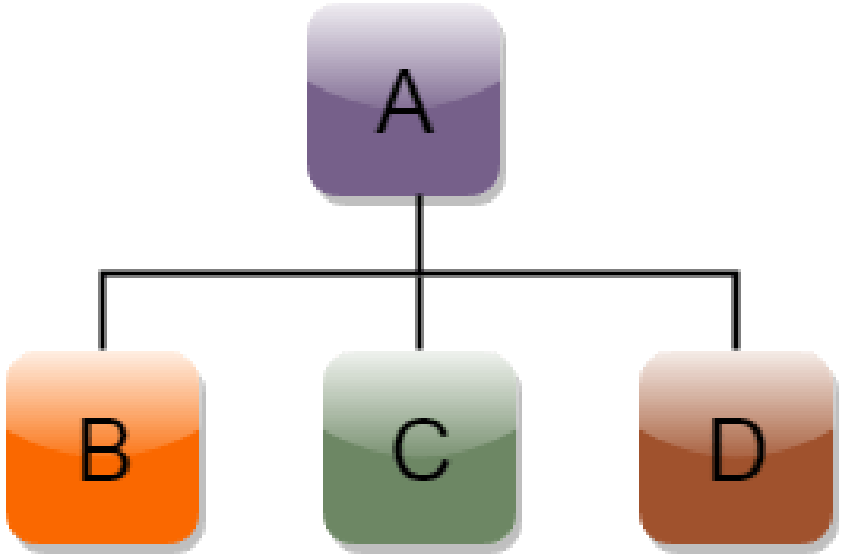
C++ Multiple Inheritance

Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes.



C++ Hierarchical Inheritance

Hierarchical inheritance is defined as the process of deriving more than one class from a base class.




```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```

```
#include <iostream>
using namespace std;
class A
{
   protected:
    int a;
   public:
   void get_a(int n)
   {
      a = n;
   }
};
   class B
   {
      protected:
      int b;
      public:
      void get_b(int n)
      {
         b = n;
      }
   };
class C : public A, public B
{
  public:
   void display()
   {
      cout << "The value of a is : " <<a<< endl;
     cout << "The value of b is : " <<b<<endl;
      cout<<"Addition of a and b is : "<<a+b;
   }
};

 int main()
 {
    C c;
    c.get_a(10);
    c.get_b(20);
    c.display();

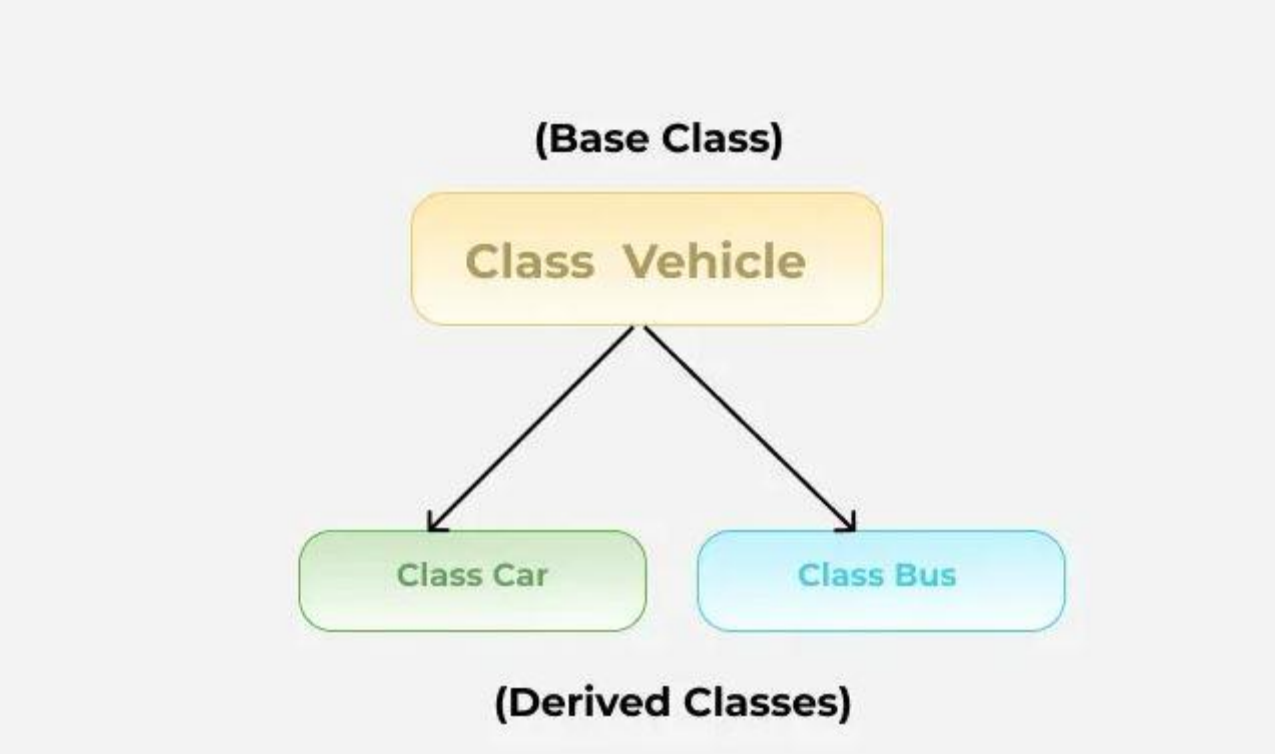
     return 0;
 }
```

Output:

```
The value of a is : 10
The value of b is : 20
Addition of a and b is : 30
```

Hierarchical Inheritance

In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class. For example, cars and buses both are vehicle.



Hierarchical Inheritance

```cpp
#include <bits/stdc++.h>
using namespace std;

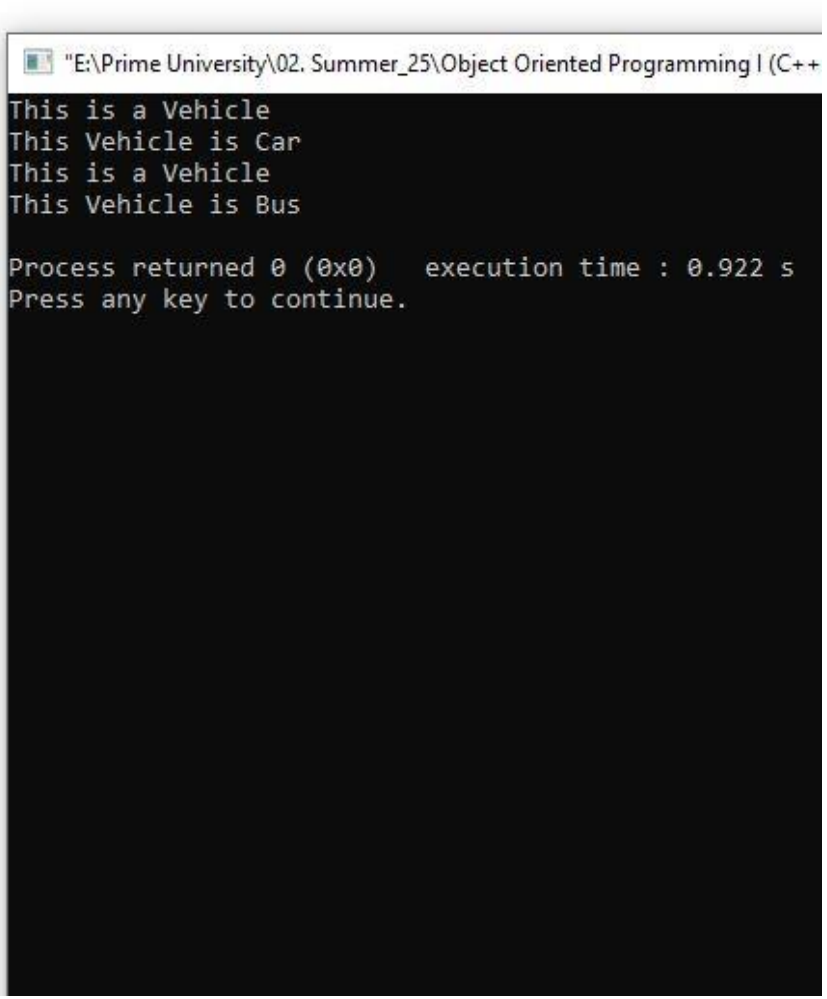
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

class Car : public Vehicle {
public:
    Car() {
        cout << "This Vehicle is Car"<< endl;
    }
};

class Bus : public Vehicle {
public:
    Bus() {
        cout << "This Vehicle is Bus"<< endl;
    }
};

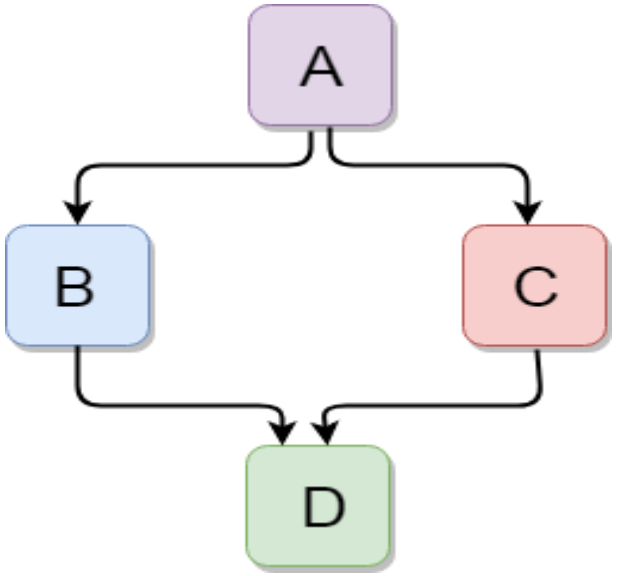
int main() {

    Car obj1;
    Bus obj2;
    return 0;
}
```



C++ Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.



Hybrid Inheritance

```cpp
#include <bits/stdc++.h>
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

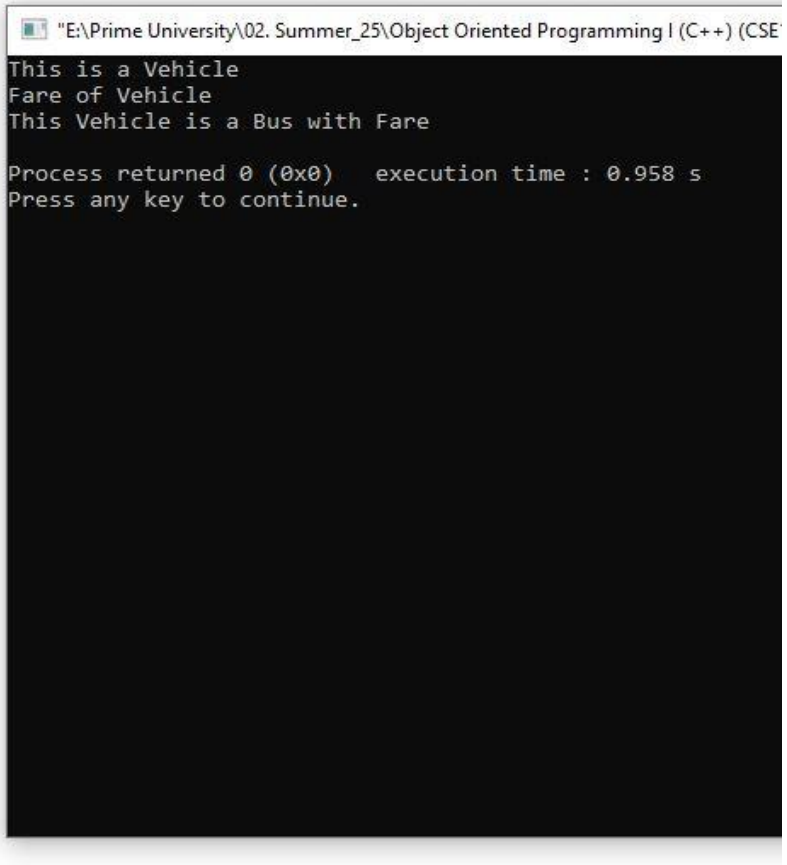
class Fare {
public:
    Fare() {
        cout << "Fare of Vehicle"<< endl;
    }
};

class Car : public Vehicle {
  public:
  Car() {
      cout << "This Vehical is a Car"<< endl;
  }
};

class Bus : public Vehicle, public Fare {
  public:
  Bus() {
      cout << "This Vehicle is a Bus with Fare" << endl;
  }
};

int main() {

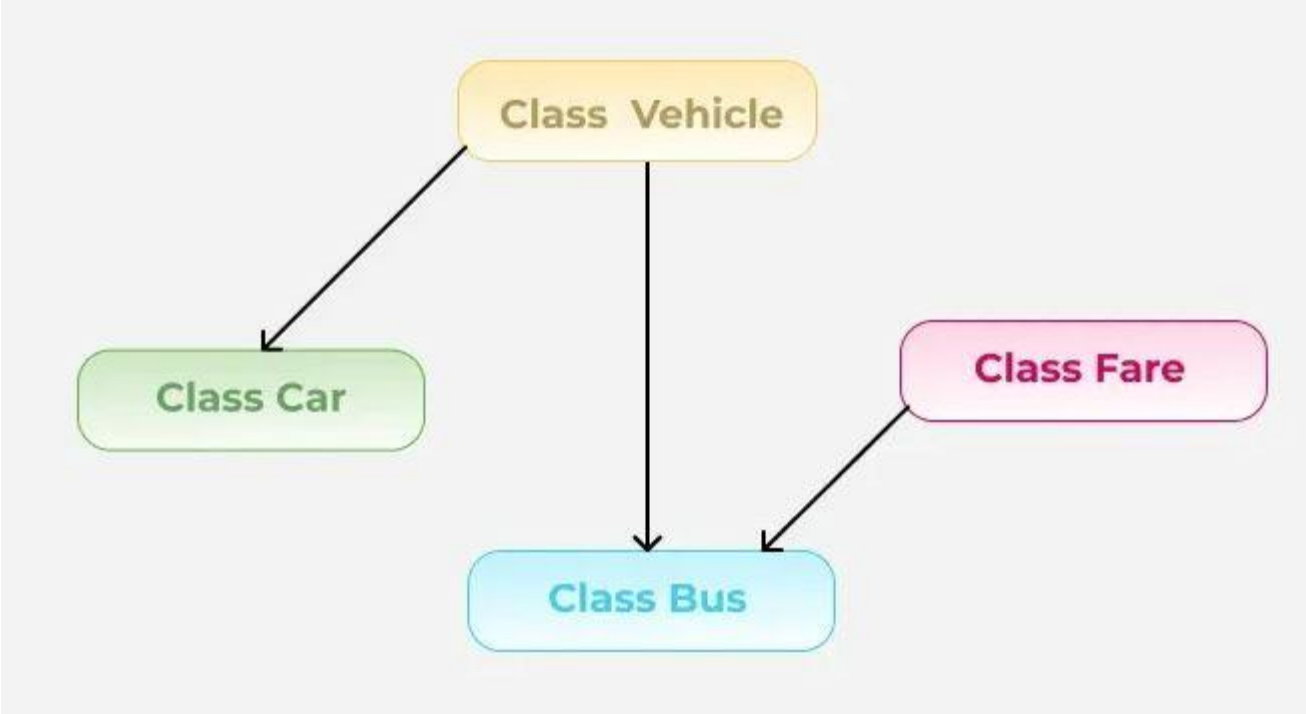
    Bus obj2;
    return 0;
}
```



Hybrid Inheritance

Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance will create hybrid inheritance in C++.

There is no particular syntax of hybrid inheritance. We can just combine two of the above inheritance types. Below image shows one of the combinations of hierarchical and multiple inheritances:



```
#include <iostream>
using namespace std;
class A
{
   protected:
   int a;
   public:
   void get_a()
   {
      cout << "Enter the value of 'a' : " << endl;
      cin>>a;
   }
};
class B :  virtual public A
{
   protected:
   int b;
   public:
   void get_b()
   {
       cout << "Enter the value of 'b' : " << endl;
      cin>>b;
   }
};
class C  : virtual  public A
{
   protected:
   int c;
   public:
   void get_c()
   {
      cout << "Enter the value of c is : " << endl;
      cin>>c;
   }
};
class D : public B, public C
{
   protected:
   int d;
   public:
   void mul()
   {
        get_a();
        get_b();
        get_c();
         cout << "Multiplication of a,b,c is : " <<a*b*c<< e

   }
```

```
#include <iostream>
using namespace std;
class Shape                 // Declaration of base class.
{
   public:
   int a;
   int b;
   void get_data(int n,int m)
   {
      a= n;
      b = m;
   }
};

class Rectangle : public Shape  // inheriting Shape class
{
   public:
   int rect_area()
   {
      int result = a*b;
      return result;
   }
};
```

```
class Triangle : public Shape    // inheriting Shape class
{
   public:
   int triangle_area()
   {
      float result = 0.5*a*b;
      return result;
   }
};
```

Thank You